

GUIA DEFINITIVO PARA PROVA DE RAZOR PAGES (.NET)

Material de revisão completo para prova teórica e prática de Programação .NET com Razor Pages. Focado em quem ainda tem pouca experiência com HTML e Razor.

1. O que é Razor Pages?

Razor Pages é uma tecnologia do ASP.NET Core usada para criar aplicações web. Cada página normalmente possui dois arquivos:

- Nome.cshtml → Frontend (HTML + Razor)
- Nome.cshtml.cs → Backend (C#)

A prova costuma cobrar a comunicação entre esses dois arquivos.

2. Arquitetura que você deve decorar

```
Pagina.cshtml
Pagina.cshtml.cs

Frontend <-----> Backend

HTML/Razor      C#
```

3. HTML ESSENCIAL

Você não precisa decorar HTML inteiro. Apenas as tags abaixo.

```
<h1> Título </h1>
<p> Texto </p>

<table>
<tr>
<td> Coluna </td>
</tr>
</table>

<form method="post">
</form>

<input />
<button>Enviar</button>
```

4. Diretivas obrigatórias

```
@page
@model NomeProjeto.Pages.FuncionariosModel
```

Sem essas diretivas a página normalmente não funciona corretamente.

5. O que é PageModel?

É a classe principal do backend da página.

```
public class FuncionariosModel : PageModel
{
}
```

6. OnGet()

Executa quando a página é aberta.

```
public void OnGet()
{
    // carregar dados
}
```

7. OnPost()

Executa quando o usuário envia um formulário.

```
public void OnPost()
```

```

{
    // processar dados
}

```

8. Classes que costumam cair

```

public class Funcionario
{
    public string Nome { get; set; }
    public string Cargo { get; set; }
    public double Salario { get; set; }
}
public class SocioTorcedor
{
    public string Nome { get; set; }
    public string Categoria { get; set; }
}

```

9. Lista (List)

```

public List<Funcionario> Funcionarios { get; set; }
Funcionarios = new List<Funcionario>();

```

10. Inserindo dados

```

Funcionarios.Add(
    new Funcionario
    {
        Nome = "João",
        Cargo = "Desenvolvedor",
        Salario = 5000
    }
);

```

11. Exercício Funcionários Completo

```

public class FuncionariosModel : PageModel
{
    public List<Funcionario> Funcionarios { get; set; }

    public void OnGet()
    {
        Funcionarios = new List<Funcionario>
        {
            new Funcionario
            {
                Nome="João",
                Cargo="Dev",
                Salario=5000
            },
            new Funcionario
            {
                Nome="Maria",
                Cargo="Gerente",
                Salario=8500
            }
        };
    }
}

```

12. Foreach (MUITO COBRADO)

```

@foreach(var funcionario in Model.Funcionarios)
{
    <tr>
        <td>@funcionario.Nome</td>
        <td>@funcionario.Cargo</td>
        <td>@funcionario.Salario</td>
    </tr>
}

```

Erro clássico de prova: esquecer a chave de fechamento do foreach.

13. Formatação de moeda

```
@funcionario.Salario.ToString("C")
```

Exibe R\$ 5.000,00.

14. ViewData

```
ViewData["Title"] = "Painel de Funcionários";  
<h2>@ViewData["Title"]</h2>
```

15. Encapsulamento

Serve para impedir valores inválidos.

```
private int _idade;  
  
public int Idade  
{  
    get => _idade;  
  
    set  
    {  
        if(value < 0)  
            _idade = 0;  
        else  
            _idade = value;  
    }  
}
```

16. Mensalidade mínima

```
private double _mensalidade;  
  
public double Mensalidade  
{  
    get => _mensalidade;  
  
    set  
    {  
        if(value < 45)  
            _mensalidade = 45;  
        else  
            _mensalidade = value;  
    }  
}
```

17. IF no Razor

```
@if(item.Categoria == "Ouro")  
{  
    <strong>@item.Nome</strong>  
}
```

18. BindProperty

Liga os inputs do formulário ao backend.

```
[BindProperty]  
public string Nome { get; set; }  
  
[BindProperty]  
public int Idade { get; set; }
```

19. asp-for

```
<input asp-for="Nome" />  
<input asp-for="Idade" />
```

20. Exercício Verificador de Acesso

```
public void OnPost()  
{  
    if(Idade >= 18)  
    {  
        MensagemDeRetorno =  
            "Entrada Liberada";  
    }  
    else
```

```

        {
            MensagemDeRetorno =
                "Entrada Bloqueada";
        }
    }
}

```

21. Exibindo mensagens

```

@if(!string.IsNullOrEmpty(Model.MensagemDeRetorno))
{
    <p>@Model.MensagemDeRetorno</p>
}

```

22. Try-Catch

```

try
{
    Resultado = Numero1 / Numero2;
}
catch(DivideByZeroException)
{
    MensagemDeErro =
        "Erro: Não é possível dividir por zero!";
}

```

23. Perguntas teóricas que podem cair

• O que é Razor Pages? • O que faz o OnGet? • O que faz o OnPost? • Para que serve PageModel? • O que é encapsulamento? • O que faz o foreach? • O que é BindProperty? • Para que serve try-catch? • O que acontece ao dividir por zero? • Diferença entre frontend e backend?

24. Checklist da prova prática

1. Criar classe. 2. Criar lista. 3. Herdar de PageModel. 4. Preencher lista no OnGet. 5. Criar @page. 6. Criar @model. 7. Fazer tabela. 8. Fazer foreach. 9. Fazer if quando solicitado. 10. Formatar moeda. 11. Conferir maiúsculas e minúsculas. 12. Conferir fechamento de chaves.

25. Resumo Final de 1 Minuto

Frontend = .cshtml Backend = .cshtml.cs OnGet = abre página OnPost = envia formulário @Model = acessa backend @foreach = repetição @if = condição PageModel = classe da página BindProperty = conecta formulário asp-for = conecta input try-catch = trata erro ToString('C') = moeda Case Sensitive = Nome != nome